

Interpretabilnost modela - Grad-CAM

Finalni izveštaj

I. Uvod

Razvoj deep learning-a doveo je do značajnog napretka u oblasti computer vision-a, posebno u zadacima klasifikacije i prepoznavanja objekata na slikama. Konvolucione neuronske mreže (CNN – Convolutional Neural Networks) pokazale su veoma dobre rezultate u ovim oblastima zahvaljujući sposobnosti automatskog izdvajanja relevantnih karakteristika iz ulaznih podataka. Međutim, sa povećanjem složenosti modela postaje sve teže razumeti na osnovu kojih informacija neuronska mreža donosi određenu odluku. Zbog toga se ovakvi modeli često posmatraju kao sistemi tipa „crne kutije“.

Potreba za boljim razumevanjem ponašanja modela dovela je do razvoja oblasti objašnjive veštačke inteligencije (XAI – Explainable Artificial Intelligence). Cilj XAI metoda jeste da korisniku omoguće uvid u način na koji model dolazi do određenog rezultata, bez potrebe za promenom njegove osnovne arhitekture ili značajnim narušavanjem performansi.

Jedna od najpoznatijih metoda za vizuelno objašnjavanje odluka konvolucionih neuronskih mreža jeste Grad-CAM (Gradient-weighted Class Activation Mapping). Ova metoda koristi gradijente ciljne klase koji prolaze kroz poslednji konvolucionni sloj mreže kako bi odredila značaj pojedinačnih mapa karakteristika za posmatranu klasu. Na osnovu dobijenih težinskih faktora formira se mapa aktivacije koja pokazuje regione ulazne slike koji su imali najveći uticaj na odluku modela.

Grad-CAM predstavlja post-hoc, lokalnu i atribucionu metodu objašnjavanja, jer se primenjuje nakon treniranja modela, objašnjava pojedinačnu predikciju i određuje doprinos pojedinih prostornih regiona ulaza rezultatu modela. Dobijena mapa može se prikazati preko originalne slike, čime se omogućava intuitivna vizuelna analiza ponašanja neuronske mreže.

U okviru ovog rada prikazana je praktična primena Grad-CAM metode na modelu za klasifikaciju slika. Pored same predikcije modela, analiziraju se aktivacije konvolucionih

slojeva i generišu Grad-CAM vizualizacije, sa ciljem da se utvrdi na koje delove ulazne slike se model oslanja prilikom donošenja odluke. Na taj način se, pored rezultata klasifikacije, dobija dodatni uvid u način rada korišćenog modela.

II. Materijali i metode

Za praktičnu analizu Grad-CAM metode korišćen je pretrenirani konvolucioni neuronski model ResNet-18. Implementacija je izvršena u programskom jeziku Python korišćenjem PyTorch okruženja, dok su za obradu slika, numeričke operacije i vizualizaciju korišćene biblioteke Torchvision, PIL, NumPy i Matplotlib. Za dodatnu proveru ručno implementirane Grad-CAM metode korišćena je biblioteka Captum i njena implementacija LayerGradCam.

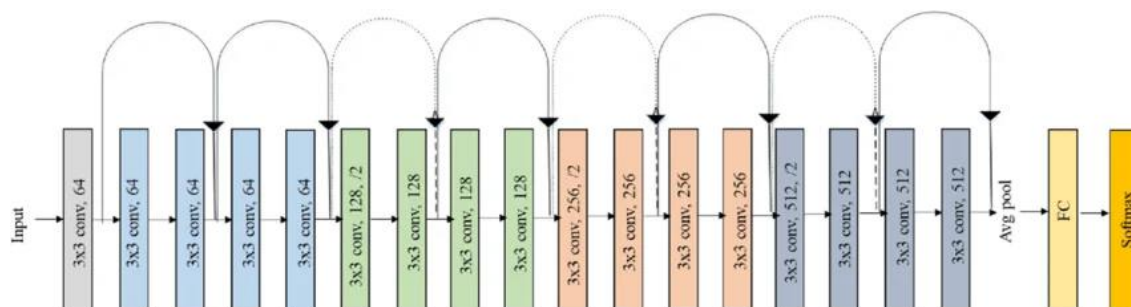
Cilj praktičnog dela bio je da se matematički postupak Grad-CAM metode direktno poveže sa njegovom programskom implementacijom. Iz tog razloga metoda je najpre implementirana ručno, odnosno eksplicitnim izdvajanjem aktivacija i gradijenata iz odabranog konvolucionog sloja mreže. Dobijeni rezultat je zatim upoređen sa implementacijom dostupnom u okviru Captum biblioteke.

III.1. Model ResNet-18

Za klasifikaciju ulaznih slika korišćen je ResNet-18 (Residual Network) model sa pretreniranim težinama. Osnovna karakteristika ResNet arhitekture jeste korišćenje rezidualnih blokova i direktnih, odnosno skip, veza. Osnovni princip rezidualnog bloka može se predstaviti izrazom:

$$y = F(x) + x,$$

gde x predstavlja ulaz u rezidualni blok, $F(x)$ transformaciju realizovanu slojevima bloka, a y njegov izlaz.



Za potrebe Grad-CAM metode posebno su značajni konvolucionni slojevi mreže, budući da njihove mape karakteristika zadržavaju prostornu informaciju o ulaznoj slici. Kao ciljni sloj za Grad-CAM analizu odabran je poslednji konvolucionni sloj poslednjeg rezidualnog bloka ResNet-18 mreže:

```
model.layer4[1].conv2
```

```
(layer4): Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (downsample): Sequential(
      (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    )
  )
  (1): BasicBlock(
    (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  )
)
```

Izabrani sloj generiše 512 mapa karakteristika prostornih dimenzija 7×7 . Izbor sloja pri kraju mreže omogućava korišćenje karakteristika višeg semantičkog nivoa uz istovremeno očuvanje prostorne informacije potrebne za lokalizaciju relevantnih regiona slike.

Model je učitán korišćenjem pretreniranih težina dostupnih u okviru biblioteke Torchvision:

```
weights = models.ResNet18_Weights.DEFAULT
model = models.resnet18(weights=weights)
model = model.to(device)
model.eval()
```

III.2. Ulazni podaci i pretprocesiranje

Za analizu ponašanja Grad-CAM metode korišćene su tri slike različite složenosti. Prva slika sadrži jasno izdvojen objekat, kakadua, druga predstavlja složeniju scenu sa većim brojem ljudi, dok treća sadrži automobil kao još jedan primer jasno prepoznatljivog objekta. Na ovaj način omogućeno je poređenje ponašanja metode na jednostavnijim i složenijim ulaznim scenama.



Pre prosleđivanja modelu slike su pretprocesirane u skladu sa zahtevima korišćenih pretreniranih ResNet-18 težina. Transformacije su preuzete direktno iz objekta koji sadrži težine modela:

```
preprocess = weights.transforms()
input_tensors = {
    name: preprocess(image).unsqueeze(0).to(device)
    for name, image in images.items()
}
```

Pretprocesiranje obuhvata promenu veličine slike, izdvajanje centralnog dela veličine 224x224 piksela, konverziju slike u tenzor i normalizaciju RGB kanala. Nakon pretprocesiranja pojedinačna slika predstavljena je tenzorom dimenzija 3x224x224, dok se dodavanjem batch dimenzije dobija ulazni tenzor oblika:

$$1 \times 3 \times 224 \times 224$$

III.3. Predikcija i izbor ciljne klase

Pretprocesirana ulazna slika prosleđuje se ResNet-18 modelu. Izlaz modela predstavlja vektor skorova za 1000 ImageNet klasa:

$$y = y[y^1, y^2, \dots, y^{1000}]$$

gde y^c predstavlja izlazni skor klase c .

Kao ciljna klasa za Grad-CAM analizu odabrana je klasa kojoj model dodeljuje najveći izlazni skor:

```
output = model(input_tensor)
target_class = output.argmax(dim=1).item()
target_score = output[0, target_class]
```

Na ovaj način se dobija skalarna veličina y^c , odnosno `target_score`, čiji će se gradijent u narednim koracima koristiti za određivanje uticaja pojedinačnih mapa karakteristika na odluku modela.

```
Cockatoo: sulphur-crested cockatoo (99.97%)
Crowd: steel drum (6.30%)
Car: tow truck (21.35%)
```

III.4. Izdvajanje aktivacija ciljnog konvolucionog sloja

Za formiranje Grad-CAM mape neophodno je izdvojiti aktivacije odabranog konvolucionog sloja. Izlaz ciljnog sloja može se predstaviti skupom mapa karakteristika

$$A = \{A^1, A^2, \dots, A^k\}$$

gde A^k predstavlja k -tu mapu karakteristika. Za odabrani sloj ResNet-18 modela dobijaju se 512 mapa karakteristika dimenzija 7×7 , odnosno tenzor oblika

$$A \in \mathbb{R}^{1 \times 512 \times 7 \times 7}$$

Za izdvajanje aktivacija bez izmene arhitekture mreže korišćen je PyTorch mehanizam forward hook. Funkcija registrovana na ciljnom sloju automatski se izvršava prilikom prolaska podataka unapred kroz mrežu i čuva njegov izlaz:

```
activations = {}
def save_activations(module, inputs, output):
    activations["value"] = output
    output.retain_grad()
hook_handle =
target_layer.register_forward_hook(save_activations)
```

Pozivom `retain_grad()` omogućava se čuvanje gradijenta izlaznog tenzora ciljnog sloja, koji je potreban u narednom koraku Grad-CAM algoritma.

III.5. Izračunavanje gradijenata ciljne klase

Nakon prolaska ulazne slike kroz mrežu dobija se skor y^c za odabranu ciljnu klasu c . Potrebno je odrediti kako promena svake vrednosti unutar mapa karakteristika utiče na ovaj skor. Ova zavisnost opisana je parcijalnim izvodom

$$\frac{\partial y^c}{\partial A_{ij}^k}$$

Gde A_{ij}^k predstavlja aktivaciju na prostornoj poziciji (i, j) u k -toj mapu karakteristika.

U implementaciji se gradijenti dobijaju propagacijom unazad, počevši od skora ciljne klase:

```
target_score.backward()
feature_maps = activations["value"]
gradients = feature_maps.grad
```

Na ovaj način dobijeni tenzor gradients ima iste dimenzije kao tenzor mapa karakteristika,

$$1 \times 512 \times 7 \times 7$$

Svaki element ovog tenzora predstavlja vrednost

$$\frac{\partial y^c}{\partial A_{ij}^k}$$

odnosno informaciju o tome kako promena odgovarajuće aktivacije utiče na skor ciljne klase.

III.6. Izračunavanje Grad-CAM težina

Gradijenti dobijeni u prethodnom koraku sadrže informaciju o značaju svake prostorne pozicije unutar pojedinačne mape karakteristika. Grad-CAM ovu informaciju svodi na jedan težinski koeficijent za svaku mapu karakteristika.

Težina k -te mape za ciljnu klasu c definisana je izrazom

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

gde Z predstavlja broj prostornih elemenata mape karakteristika. Kako korišćene mape imaju dimenzije 7×7 , u ovom slučaju važi

$$Z = 7 * 7 = 49$$

Praktično, vrši se globalno usrednjavanje gradijenata (Global Average Pooling) po obe prostorne dimenzije:

```
weights_gradcam = gradients.mean(  
    dim=(2, 3),  
    keepdim=True  
)
```

Od početnog tenzora gradijenata dimenzija $1 \times 512 \times 7 \times 7$ dobija se tenzor težina dimenzija $1 \times 512 \times 1 \times 1$. Prema tome, svakoj od 512 mapa karakteristika pridružena je jedna vrednost α_k^c . Veća pozitivna vrednost težine ukazuje na veći pozitivan doprinos odgovarajuće mape skoru posmatrane klase.

III.7. Formiranje Grad-CAM lokalizacione mape

Nakon određivanja težinskih koeficijenata, svaka mapa karakteristika A^k množi se odgovarajućom težinom α_k^c . Ponderisane mape zatim se sabiraju po svim kanalima:

$$S^c = \sum_k \alpha_k^c A^k$$

U programskoj implementaciji ove operacije realizovane su na sledeći način:

```
weighted_feature_maps = weights_gradcam * feature_maps
gradcam = weighted_feature_maps.sum(
    dim=1,
    keepdim=True
)
```

Na dobijenu mapu primenjuje se ReLU (Rectified Linear Unit) funkcija:

$$L_{Grad-CAM}^c = ReLU(\sum_k \alpha_k^c A^k)$$

U kodu ovaj korak odgovara naredbi:

```
gradcam = torch.relu(gradcam)
```

Primena ReLU funkcije uklanja negativne vrednosti i zadržava regione koji pozitivno doprinose skorcu ciljne klase. Kao rezultat dobija se klasno-specifična Grad-CAM lokalizaciona mapa dimenzija

$$1 \times 1 \times 7 \times 7$$

Na ovaj način ostvarena je direktna veza između matematičke formulacije Grad-CAM metode i njene programske implementacije.

III.8. Interpolacija, normalizacija i vizualizacija Grad-CAM mape

Grad-CAM mapa dobijena na izlazu odabranog konvolucionog sloja ima prostorne dimenzije 7×7 , dok ulazna slika modela ima dimenzije 224×224 piksela. Da bi se dobijena mapa mogla prostorno povezati sa ulaznom slikom, potrebno je povećati njenu rezoluciju.

```
gradcam = F.interpolate(
    gradcam,
    size=(224, 224),
    mode="bilinear",
    align_corners=False
)
```

Bilinearnom interpolacijom Grad-CAM mapa se transformiše iz 7×7 u 224×224 piksela. Interpolacijom se ne dobija nova informacija o odluci modela, već se postojeća lokalizaciona mapa preslikava na rezoluciju pogodnu za poređenje sa ulaznom slikom. Zbog relativno male prostorne rezolucije poslednjeg konvolucionog sloja, Grad-CAM prvenstveno daje grubu lokalizaciju relevantnih regiona, a ne preciznu segmentaciju objekta.

Nakon interpolacije izvršena je normalizacija vrednosti Grad-CAM mape. Minimalna vrednost se oduzima od svih elemenata, nakon čega se rezultat deli maksimalnom vrednošću:

```
gradcam = gradcam - gradcam.min()
if gradcam.max() > 0:
    gradcam = gradcam / gradcam.max()
```

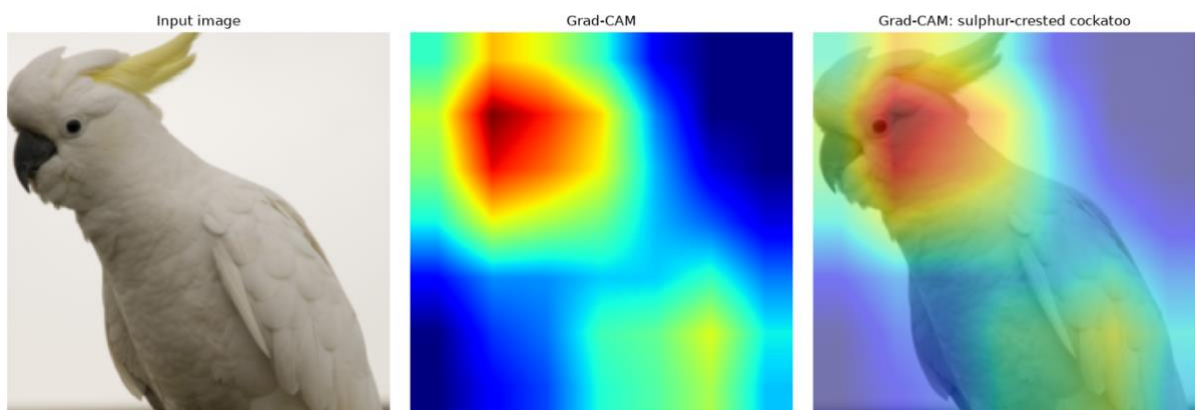
Ovim postupkom vrednosti Grad-CAM mape svode se na interval

$$L_{Grad-CAM}^c(i,j) \in [0,1]$$

Vrednosti bliže jedinici označavaju regione sa većim pozitivnim doprinosom predikciji posmatrane klase, dok vrednosti bliže nuli označavaju regione sa manjim doprinosom.

Radi intuitivnijeg prikaza, normalizovana mapa prikazuje se kao toplotna mapa (heatmap) i postavlja preko originalne slike. Na taj način moguće je neposredno uočiti regione ulazne slike koji su imali najveći uticaj na odluku modela.

```
plt.imshow(image)
plt.imshow(
    gradcam,
    cmap="jet",
    alpha=0.5
)
plt.axis("off")
```



Parametar alpha određuje transparentnost Grad-CAM mape, čime se istovremeno omogućava prikaz originalnog sadržaja slike i regiona koje je model smatrao značajnim. Rešenje prikazano isto i pomoću Captum biblioteke, te su razltati ručnog pristupa i pomenutog upoređjeni:



Konačni rezultat ovog postupka predstavlja klasno-specifičnu vizuelnu interpretaciju predikcije ResNet-18 modela. Time se kompletan postupak može predstaviti kao

*ulazna slika → mape karakteristika → gradijenti → Grad – CAM mapa
→ interpolacija i normalizacija → vizualno objašnjenje*

III. Rezultati i diskusija

Primenom Grad-CAM metode na prethodno opisani ResNet-18 model dobijene su klasno-specifične toplotne mape koje prikazuju regione ulazne slike sa najvećim uticajem na posmatranu predikciju. Analiza je sprovedena na slikama različite složenosti, čime je bilo moguće posmatrati ponašanje metode kako u slučaju uspešne klasifikacije, tako i u slučajevima u kojima predikovana klasa ne odgovara očekivanom sadržaju slike.

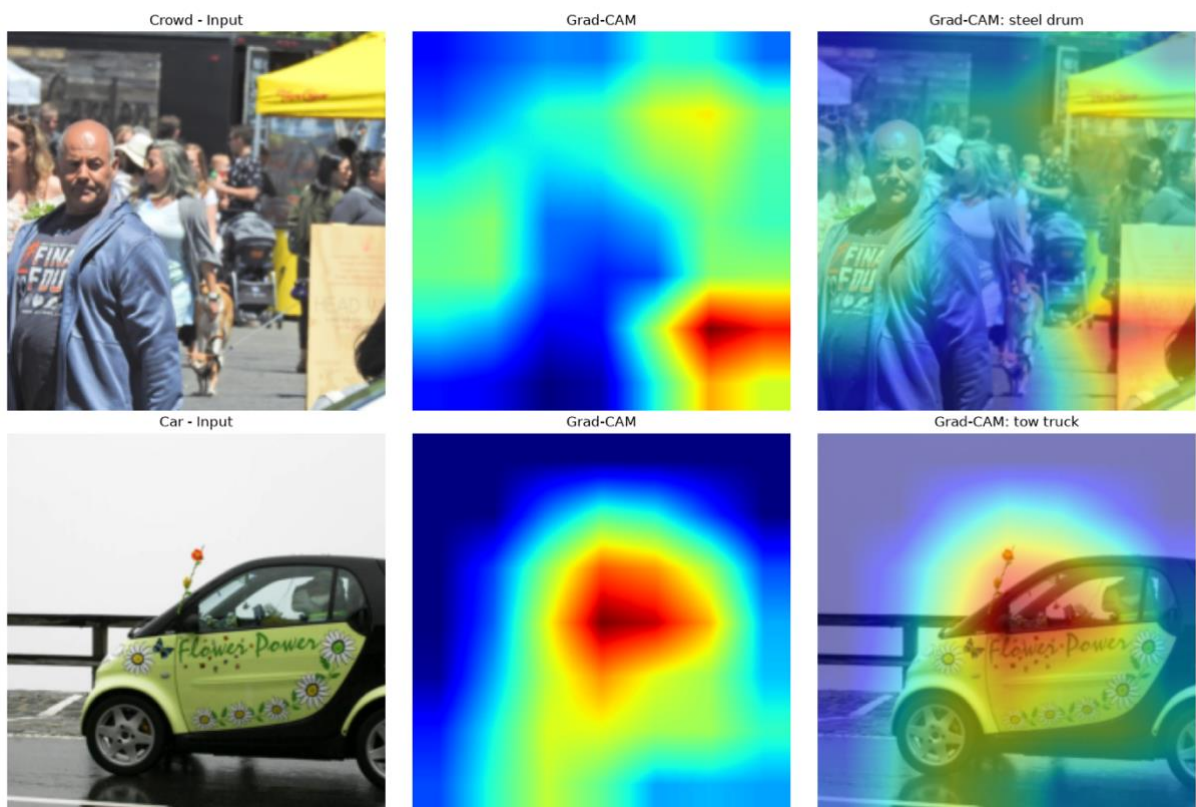
Ovakav način analize odgovara jednoj od osnovnih namena Grad-CAM metode predstavljenih u originalnom radu. Autori navode da Grad-CAM omogućava uvid u greške postojećih CNN klasifikatora i pokazuju da čak i naizgled neočekivane predikcije mogu imati razumljivo vizuelno objašnjenje kada se posmatraju regioni slike koji su uticali na odluku modela.

III.1. Analiza dobijenih rezultata

Na slici kakadua ResNet-18 model uspešno prepoznaje prikazani objekat. Grad-CAM vizualizacija pokazuje da su regioni sa najvećim intenzitetom koncentrisani prvenstveno na prostoru slike koji zauzima ptica. U ovom slučaju postoji očekivano poklapanje između predikcije modela, stvarnog sadržaja slike i regiona koje Grad-CAM označava kao značajne.



Zanimljiviji rezultat dobijen je na složenijoj slici koja prikazuje grupu ljudi. Kao najverovatniju klasu ResNet-18 model predvideo je steel drum. Sama informacija o predikovanj klasi ne daje dovoljno podataka da se utvrdi zbog čega je mreža donela takvu odluku. Grad-CAM mapa, međutim, omogućava da se posmatraju regioni slike koji su najviše doprineli skoru klase steel drum.



Ovaj primer je posebno značajan sa aspekta objašnjive veštačke inteligencije. Grad-CAM ne pokušava da ispravi pogrešnu predikciju, već omogućava analizu ponašanja modela koje je do takve predikcije dovelo. Upravo ovakvu primenu autori originalnog rada izdvajaju kao jedan od načina za dijagnostikovanje grešaka CNN klasifikatora.

Sličan rezultat dobijen je na slici automobila, gde je ResNet-18 kao najverovatniju klasu predvideo tow truck. Za razliku od prethodnog primera, ovde pogrešno predviđena klasa pripada semantički i vizuelno srodnoj kategoriji, odnosno takođe predstavlja vozilo. Posmatranjem Grad-CAM mape moguće je proveriti da li su visoke aktivacije koncentrisane na samom automobilu ili na elementima njegove okoline.

Dobijeni rezultati pokazuju važnu razliku između rezultata klasifikacije i njegovog objašnjenja. Predikovana klasa daje odgovor na pitanje šta je model predvideo, dok Grad-CAM vizualizacija pruža dodatnu informaciju o tome koji regioni slike su doprineli toj predikciji. Zbog toga metoda može biti korisna i kada je predikcija ispravna i kada model napravi grešku.

III.2. Interpretabilnost Grad-CAM vizualizacija

Jedna od osnovnih karakteristika Grad-CAM metode jeste klasna diskriminativnost. Mapa se ne formira samo na osnovu aktivacija konvolucionog sloja, već na osnovu njihovog značaja za konkretnu ciljnu klasu. Zbog toga se za istu ulaznu sliku mogu dobiti različite mape u zavisnosti od klase čiji se skor analizira.

U originalnom radu interpretabilnost metode ispitivana je i kroz eksperimente sa korisnicima, dok je vernost vizualizacije ponašanju modela analizirana poređenjem sa drugim metodama. Autori definišu vernost (faithfulness) kao sposobnost vizualizacije da pravilno objasni funkciju koju je model naučio, pri čemu ukazuju i na kompromis između potpune vernosti modelu i jednostavnosti interpretacije objašnjenja.

Kao jedan od načina provere korišćena je metoda zaklanjanja delova slike (image occlusion). Poređenjem je pokazano da regioni čije zaklanjanje značajno menja izlazni skor modela često odgovaraju regionima kojima Grad-CAM dodeljuje visok intenzitet. U eksperimentima predstavljenim u radu Grad-CAM je ostvario veću korelaciju sa occlusion mapama od više prethodno korišćenih metoda vizualizacije.

Ovakvi rezultati su značajni za interpretaciju vizualizacija dobijenih u ovom radu. Visok intenzitet Grad-CAM mape ne treba tumačiti kao tvrdnju da se na tom mestu nužno nalazi objekat određene klase. Umesto toga, on ukazuje da su karakteristike izdvojene u tom regionu bile značajne za izlaz modela za analiziranu klasu.

III.3. Prednosti i ograničenja metode

Važna prednost Grad-CAM metode jeste mogućnost primene na već obučene CNN modele bez potrebe za promenom njihove arhitekture ili ponovnim treniranjem. Upravo ovu osobinu autori originalnog rada navode kao jednu od ključnih karakteristika metode.

Druga značajna prednost jeste intuitivan način prikaza rezultata. Preklapanjem toplotne mape sa originalnom slikom moguće je neposredno povezati izlaz modela sa određenim prostornim regionima. Primeri pogrešnih predikcija steel drum i tow truck pokazuju

praktičnu vrednost takvog pristupa: pogrešna izlazna klasa sama po sebi pruža malo informacija o uzroku greške, dok Grad-CAM omogućava dodatnu analizu karakteristika na koje je model reagovao.

Autori originalnog rada takođe pokazuju da ovakve vizualizacije mogu biti korisne za analizu pouzdanosti ponašanja klasifikatora i identifikovanje pristrasnosti prisutnih u skupovima podataka.

Sa druge strane, značajno ograničenje Grad-CAM metode predstavlja prostorna rezolucija dobijenih mapa. U sprovedenoj implementaciji Grad-CAM mapa se formira na osnovu poslednjeg konvolucionog sloja ResNet-18 modela, čije mape karakteristika imaju dimenzije 7×7 . Njihova naknadna interpolacija na 224×224 piksela omogućava prikaz preko originalne slike, ali ne stvara novu prostornu informaciju. Zbog toga dobijene vizualizacije predstavljaju približnu lokalizaciju značajnih regiona, a ne preciznu segmentaciju granica objekta.

Takođe, Grad-CAM objašnjava ponašanje analiziranog modela, a ne objektivnu ispravnost njegove odluke. Ukoliko model koristi neočekivane karakteristike slike ili napravi pogrešnu klasifikaciju, Grad-CAM može pomoći u njihovom uočavanju, ali sam po sebi ne menja niti ispravlja ponašanje modela.

Rezultati originalnog rada pokazuju da Grad-CAM vizualizacije mogu pružiti korisne informacije o klasnoj diskriminativnosti, vernosti objašnjenja modelu i potencijalnim greškama ili pristrasnostima modela. Rezultati dobijeni u ovom radu ilustruju istu osnovnu ideju na ResNet-18 modelu: Grad-CAM omogućava da se pored same predikcije posmatra i prostorni region slike koji je doprineo njenom formiranju, što je naročito korisno pri analizi neočekivanih i pogrešnih klasifikacija.

IV. Zaključak

U ovom radu predstavljena je Grad-CAM metoda za vizuelno objašnjavanje odluka konvolucionih neuronskih mreža. Metoda je analizirana kroz njenu matematičku osnovu, a zatim je praktično implementirana na prethodno obučenom ResNet-18 modelu. Grad-CAM je najpre realizovan ručno, čime je omogućeno direktno povezivanje matematičkih izraza sa odgovarajućim koracima implementacije, a zatim je prikazana i primena pomoću Captum biblioteke.

Na analiziranim slikama pokazano je da Grad-CAM omogućava identifikovanje regiona koji su najviše doprineli konkretnoj predikciji modela. Posebno su značajni primeri pogrešnih predikcija, poput klasa steel drum i tow truck, kod kojih vizualizacija pruža dodatni uvid u ponašanje modela koji nije moguće dobiti posmatranjem samo izlazne klase.

Na osnovu sprovedene analize može se zaključiti da Grad-CAM predstavlja jednostavan i praktičan pristup povećanju interpretabilnosti CNN modela bez potrebe za promenom njihove arhitekture ili ponovnim treniranjem. Iako relativno mala prostorna rezolucija ograničava preciznost lokalizacije, metoda pruža koristan vizuelni uvid u proces formiranja predikcija i može se koristiti za analizu ispravnih i pogrešnih odluka modela.